Advanced Day 11 — Session 11 (Hours 41–44)

Python Programming (Advanced) • Pandas, Matplotlib, Regression, and the Capstone Report Feature

Session 11 Overview

Topics Covered Today

- Hour 41 — pandas essentials: loading, cleaning, summarizing
- Hour 42 — visualization with matplotlib: charts you can ship
- Hour 43 — regression demo: practical, limited scope
- Hour 44 — integrate analytics into the capstone report feature

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour1_Advanced.md` → `# Day 11, Hour 1: Pandas essentials: loading, cleaning, summarizing`
- `Advanced/lessons/lecture/Day11_Hour2_Advanced.md` → `# Day 11, Hour 2: Visualization with matplotlib: charts you can ship`

Session 11 Outcomes

By the end of today, learners will be able to:

- Load capstone data into a pandas DataFrame and inspect it before analysis
- Clean common data issues: missing values, numeric strings, and whitespace
- Produce summary metrics saved to `reports/summary.csv`
- Create readable, labeled charts saved to `reports/`
- Decide when a regression workflow fits — and when a fallback analysis is better
- Wire all analytics steps into a repeatable one-command report feature

Scope Guardrails for Today

In Scope

- pandas load, inspect, clean, and summarize from the capstone CSV
- matplotlib bar chart with title, labeled axes, and a saved PNG
- Simple train/test regression demo — or honest fallback analysis
- One-command report generation saving artifacts to `reports/`
- `pathlib` paths, small functions, and project-relative directories

Not Yet

- Production machine-learning pipelines or model deployment
- Deep statistical analysis or multi-model comparison
- Jupyter notebooks as the final deliverable

Hour 41 — Pandas Essentials

Loading, Cleaning, and Summarizing

Course Hour 41 of 44

Hour 41 — Learning Outcomes

By the end of this hour, learners will be able to:

- Load capstone data into a pandas DataFrame
- Run `head()`, `info()`, and `describe()` to understand the dataset
- Clean at least one numeric column and one categorical column
- Compute grouped summaries and save `reports/summary.csv`

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour1_Advanced.md` → `## Learning Outcomes`

The Analytics Pipeline Starts Here

The capstone already stores data in SQLite.
This hour adds an outward-facing view of that data.

The Session 11 pipeline:

```
export records → load into pandas → clean
  → summarize → chart → regression (if fit)
  → one-command report generation
```

- Use the **same dataset** across all four hours
- Clean data once; use it everywhere downstream

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour1_Advanced.md` → `### Talk points`

Why Inspect Before You Analyze

DataFrames can hide surprises:

Inspection call	What it reveals
`df.head()`	Column names, sample values, obvious gaps
`df.dtypes`	Whether numerics are stored as strings
`df.shape`	Row and column count
`df.isna().sum()`	Missing-value counts per column

> "Inspect before you interpret."

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour1_Advanced.md` → `### Demo steps`

Demo — Load and Inspect

```python
from pathlib import Path
import pandas as pd

DATA_PATH = Path("data/records.csv")
REPORTS_DIR = Path("reports")
REPORTS_DIR.mkdir(exist_ok=True)

df = pd.read_csv(DATA_PATH)
print(df.head())
print(df.dtypes)
print(df.describe(include="all"))
```

Happy path: prints column types and a five-row preview

Sad path: `FileNotFoundError` if `data/records.csv` does not exist

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour1_Advanced.md` → `### Demo code or command`

Demo — Clean

```python
# Coerce numeric strings to NaN, then drop unresolvable rows
df["amount"] = pd.to_numeric(df["amount"], errors="coerce")

# Fill missing categorical values with a sentinel
df["status"] = df["status"].fillna("unknown")

# Drop rows where the target column is still missing
clean_df = df.dropna(subset=["amount"])

print(f"Rows before: {len(df)}  Rows after: {len(clean_df)}")
```

Rule: explain every cleaning decision in a comment

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour1_Advanced.md` → `### Demo code or command sketch`

Demo — Summarize and Save

```
summary = (
    clean_df
    .groupby("status", as_index=False)["amount"]
    .agg(["count", "mean", "sum"])
)
summary.to_csv(REPORTS_DIR / "summary.csv")
print(summary)
```

Completion evidence:

- `reports/summary.csv` exists and is re-creatable
- Learner can explain one cleaning choice and its row-count effect

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour1_Advanced.md` → `### Demo code or command sketch`

Lab — Analytics Starter (Hour 41)

Time: 35–50 minutes

Tasks

1. Export or locate your capstone CSV under `data/`
2. Load it and run `head()`, `dtypes`, `describe()`
3. Make at least one cleaning decision — document it in a comment
4. Compute at least three metrics (count, mean, sum, or group counts)
5. Save a summary table to `reports/summary.csv`

Completion Criteria

- Data loads without error from a `pathlib` path
- At least three metrics are computed

Common Pitfalls — Hour 41

Warning: Wrong delimiter or encoding — add `sep=","` or `encoding="utf-8"` explicitly

Warning: Numeric values stored as strings — always check `dtypes` before summarizing

Warning: Dropping rows without explaining what was removed

Warning: Hard-coded absolute paths — use `pathlib.Path` relative to the project root

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour1_Advanced.md` → `### Common pitfalls to watch for`

Quick Check — Hour 41

Question: What does `df.head()` help you confirm *before* deeper analysis?

Bonus: If `df["amount"].dtype` is `object`, what is the first cleaning step?

Hour 42 — Visualization with Matplotlib

Charts You Can Ship

Course Hour 42 of 44

Hour 42 — Learning Outcomes

By the end of this hour, learners will be able to:

- Create readable charts from the cleaned capstone dataset
- Save figures as PNG files for a repeatable report workflow
- Choose chart types that match the question being answered
- Avoid common figure-management mistakes

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour2_Advanced.md` → `## Learning Outcomes`

A Chart Is a Communication Tool

Four qualities every report-ready chart must have:

1. **Title** — answers "what am I looking at?"
2. **Labeled axes** — x and y both described
3. **Readable categories** — rotation if text overlaps
4. **Saved artifact** — a PNG under `reports/`

A chart that only lives in a notebook session is not yet a deliverable.

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour2_Advanced.md` → `### Talk points`

Start from Aggregated Data

Plot the **summary**, not the raw noise:

- Counts by status → bar chart
- Counts by category → bar chart
- Records over time → line chart (if timestamps exist)
- Distribution of amounts → histogram

Practical rule: if the question is "how many?", the chart is a bar chart.

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour2_Advanced.md` → `### Demo steps`

Demo — Bar Chart with OO Interface

```
from pathlib import Path
import matplotlib.pyplot as plt
import pandas as pd

REPORTS_DIR = Path("reports")
REPORTS_DIR.mkdir(exist_ok=True)
df = pd.read_csv(Path("data/records.csv"))
counts = df["status"].fillna("unknown").value_counts().sort_index()

fig, ax = plt.subplots(figsize=(8, 4))
counts.plot(kind="bar", ax=ax)
ax.set_title("Tracker records by status")
ax.set_xlabel("Status")
ax.set_ylabel("Record count")
ax.tick_params(axis="x", rotation=30)
fig.tight_layout()
fig.savefig(REPORTS_DIR / "status_counts.png", dpi=150)
plt.close(fig)
```

Chart Type Reference

Question	Chart type
How many per category?	Bar chart
How does value change over time?	Line chart
How are values distributed?	Histogram
How do two numbers relate?	Scatter plot

Today's scope: bar chart and, optionally, histogram or line chart

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour2_Advanced.md` → `### Talk points`

Sad Path — Missing Required Columns

```
required_columns = {"status"}
missing_columns = required_columns - set(df.columns)
if missing_columns:
    raise ValueError(
        f"Chart data is missing columns: {sorted(missing_columns)}"
    )
```

Check required columns **before** plotting, not after an `AttributeError`.

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour2_Advanced.md` → `### Demo code or command sketch`

Lab — Generate Report-Ready Charts (Hour 42)

Time: 35–50 minutes

Tasks

1. Reuse the cleaned dataset from Hour 41
2. Create at least one bar chart answering a clear question
3. Add a title, x-label, and y-label
4. Save the PNG to `reports/` — confirm it opens correctly
5. Stretch: add a second chart (histogram or line)

Completion Criteria

- At least one PNG saved under `reports/`
- Chart answers a clear question about the tracker data

Common Pitfalls — Hour 42

Warning: Forgetting `plt.close(fig)` — causes overlapping charts on subsequent runs

Warning: Unlabeled axes or ambiguous titles — the chart cannot stand alone

Warning: Saving to a directory that does not exist — use `mkdir(exist_ok=True)` first

Warning: Pie chart when categories number more than five — use a bar chart instead

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour2_Advanced.md` → `### Common pitfalls to watch for`

Quick Check — Hour 42

Question: Why should a report script save plots to files instead of only displaying them?

Bonus: What is the correct order — `fig.tight_layout()` then `fig.savefig()`, or the reverse?

Hour 43 — Regression Demo

Practical, Limited Scope

Course Hour 43 of 44

Hour 43 — Learning Outcomes

By the end of this hour, learners will be able to:

- Run a simple regression workflow when the dataset supports it
- Explain train/test split and one evaluation metric in plain language
- Recognize when regression is not appropriate and choose a better analysis

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour3_Advanced.md` → `## Learning Outcomes`

When Regression Fits

Use the suitability checklist before starting:

Criterion	Required
Target is numeric	Yes
Features are numeric and clean	Yes
Enough rows for a meaningful split	Yes
Question genuinely involves prediction	Yes

If any criterion fails: produce an additional summary or chart instead.

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour3_Advanced.md` → `### Instructor Prep Notes` and `### Talk points`

Train/Test Split — Why It Matters

Training on all rows, then evaluating on those same rows, produces an **optimistic lie**.

```
all rows
   75% train  →  model.fit()
   25% test   →  model.predict()  →  metric
```

- Hold back test rows the model has never seen
- Report the metric on the **test split only**
- One metric is enough for today — explain it in plain language

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour3_Advanced.md` → `### Talk points`

Demo — Regression with scikit-learn

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split

X = model_data[["record_number"]]
y = model_data[target_column]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
print(f"MAE: {mean_absolute_error(y_test, predictions):.2f}")
```

MAE = mean absolute error — average distance between prediction and reality

Source Alignment

Demo — Fallback When scikit-learn Is Absent

```python
try:
    from sklearn.linear_model import LinearRegression
    from sklearn.metrics import mean_absolute_error
    from sklearn.model_selection import train_test_split
    # ... full regression workflow above ...
except ImportError:
    correlation = (
        model_data["record_number"]
        .corr(model_data[target_column])
    )
    print(f"scikit-learn unavailable. Correlation fallback: {correlation:.2f}")
```

Pattern: `try/except ImportError` keeps the script runnable in any environment.

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour3_Advanced.md` → `### Demo code or command sketch`

Regression Suitability Checklist

Before claiming a model is useful, answer all five questions:

1. Is the target genuinely numeric (not a coded category)?
2. Are there enough rows for a meaningful test split?
3. Are the features cleaned of NaN and string values?
4. Does the prediction question make real-world sense?
5. Can you explain the metric result in one sentence?

If the answer to any is "no" — choose the fallback analysis.

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour3_Advanced.md` → `### Instructor Prep Notes`

Lab — Regression Mini-lab (Hour 43)

Time: 35–50 minutes

Tasks

1. Decide whether your cleaned data supports regression
2. If yes: split, fit `LinearRegression`, report MAE or R^2
3. If no: produce an additional summary and one more chart
4. Write one sentence explaining the result in plain language

Completion Criteria

- Regression run or equivalent analysis completed
- Learner can explain the metric or observation
- No one claims a model is useful solely because the code ran

Common Pitfalls — Hour 43

Warning: Feeding categorical strings directly into a numeric model without encoding

Warning: Training and evaluating on the **same** rows — hides poor generalization

Warning: Uncleaned data fed into scikit-learn — garbage in, garbage out

Warning: Overstating conclusions from a tiny synthetic dataset

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour3_Advanced.md` → `### Common pitfalls to watch for`

Quick Check — Hour 43

Question: Why do we separate training data from testing data?

Bonus: Why is "regression is not the right fit here" sometimes the most advanced answer?

Hour 44 — Integrate Analytics

Capstone Report Feature

Course Hour 44 of 44

Hour 44 — Learning Outcomes

By the end of this hour, learners will be able to:

- Add a repeatable analytics/export feature to the capstone
- Generate CSV, summary, and chart artifacts with one command or button
- Document how reviewers run the report

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour4_Advanced.md` → `## Learning Outcomes`

Analytics as a Capstone Feature

A report feature is valuable when it is **repeatable**.

The pipeline:

```
export records → load + clean → summarize
  → generate charts → save all artifacts
```

Anti-pattern: hidden manual steps between stages

Pro-pattern: one command, same clean result every time

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour4_Advanced.md` → `### Talk points`

Repeatable Pipeline Design

Use `pathlib` throughout — never hard-code paths:

```
from pathlib import Path

DATA_PATH = Path("data/records.csv")
REPORTS_DIR = Path("reports")
REPORTS_DIR.mkdir(parents=True, exist_ok=True)
```

Rules:

- Create output directories in code, not by hand
- Use relative paths so the project runs on any machine
- Running the report twice should produce the same artifacts

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour4_Advanced.md` → `### Talk points`

Demo — generate_report Function

```python
from pathlib import Path
import pandas as pd

REQUIRED_COLUMNS = {"status"}

def generate_report(data_path: Path, reports_dir: Path) -> None:
    if not data_path.exists():
        raise FileNotFoundError(
            f"Report input not found: {data_path}. "
            "Export tracker records before running the report."
        )
    reports_dir.mkdir(parents=True, exist_ok=True)
    df = pd.read_csv(data_path)
    missing = REQUIRED_COLUMNS - set(df.columns)
    if missing:
        raise ValueError(f"Report CSV is missing columns: {sorted(missing)}")
    if df.empty:
        raise ValueError("Report CSV contains no records to summarize.")
    summary = df.groupby("status", as_index=False).size()
    summary.to_csv(reports_dir / "summary.csv", index=False)
```

Demo — Chart Function and Entry Point

```python
import matplotlib.pyplot as plt

def create_status_chart(summary: pd.DataFrame, output_path: Path) -> None:
    fig, ax = plt.subplots()
    ax.bar(summary["status"], summary["size"])
    ax.set_title("Records by status")
    ax.set_xlabel("Status")
    ax.set_ylabel("Count")
    fig.tight_layout()
    fig.savefig(output_path)
    plt.close(fig)

def main() -> None:
    data_path = Path("data/records.csv")
    reports_dir = Path("reports")
    generate_report(data_path, reports_dir)
    df = pd.read_csv(data_path)
    summary = df.groupby("status", as_index=False).size()
    create_status_chart(summary, reports_dir / "status_counts.png")

if __name__ == "__main__":
    main()
```

Sad Path — Missing or Empty Data

Validate early and fail with clear messages:

```
if not data_path.exists():
    raise FileNotFoundError(
        f"Report input not found: {data_path}. "
        "Export tracker records before running the report."
    )

if df.empty:
    raise ValueError("Report CSV contains no records to summarize.")
```

A clear error message is part of the feature — the reviewer experience matters.

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour4_Advanced.md` → `### Demo code or command sketch`

Lab — Report Integration (Hour 44)

Time: 35–50 minutes

Tasks

1. Create `reports/generate_report.py` with a `main()` function
2. Call the repository export or read the existing CSV
3. Generate `reports/summary.csv` and at least one PNG
4. Run `python -m reports.generate_report` twice — prove repeatability
5. Document the run command in the README

Completion Criteria

- One-command report generation works
- Artifacts are saved and documented

Common Pitfalls — Hour 44

Warning: Hard-coded absolute paths — always use `pathlib.Path` relative to the project root

Warning: Report logic scattered across scripts and notebook fragments

Warning: README says "run X" but command X no longer exists

Warning: Charts overwrite each other silently without a naming plan

Source Alignment

- `Advanced/lessons/lecture/Day11_Hour4_Advanced.md` → `### Common pitfalls to watch for`

Quick Check — Hour 44

Question: What makes a report feature *repeatable* rather than "works when I remember the steps"?

Bonus: Name one sad path the report function should handle before generating any artifacts.

Capstone Quality Gate

Category	Ready evidence	Needs attention
Correctness	Feature works with realistic data	Only works with hard-coded or demo data
Error handling	Sad path raises a clear message	App crashes or hides the cause
Structure	Logic sits in the correct layer	Service, SQL, and report logic are tangled
Repeatability	One command re-creates all artifacts	Depends on manual memory or IDE state
Maintainability	Names are clear; next feature has a home	Works once but will be hard to extend

Source Alignment

Session 11 Summary

What We Built Today

Hour	Deliverable
41 — pandas essentials	`reports/summary.csv` and at least three metrics
42 — matplotlib charts	`reports/status_counts.png` with labeled axes
43 — regression demo	MAE metric or documented fallback analysis
44 — report integration	One-command `generate_report` feature and README

Through-line: the same dataset, cleaned once, flows through every stage.

What's Next — Day 12 Preview

Session 12 — Hours 45–48

- Lock in quality with pytest: test structure, parametrize, and fixtures
- Use coverage to find untested paths
- Package the capstone for handoff: `pyproject.toml`, `build`, `install`
- Final capstone demo and reflection

Prepare before Day 12:

- Ensure `reports/` contains at least one artifact from today
- Commit today's work: `git commit -m "feat: add analytics and report pipeline"`
- Read `reports/generate_report.py` once more — tests will target its functions